

面向共享 L2 复用的 MoE 层离线 tile 调度

1. Introduction

大语言模型扩容面临一个直接矛盾：增加参数可以扩大模型容量，但若每个 token 都计算全部参数，推理开销也会随之增加。混合专家模型（Mixture-of-Experts, MoE）通过路由机制仅激活部分专家，使模型能够在扩大总参数规模的同时，控制单个 token 实际参与计算的参数量。不过，稀疏激活并未自动解决计算之外开销：路由后的输入、专家权重与中间结果如何读取、复用和组织，仍会影响实际推理效率。

MoE 层用多个专家网络替代 Transformer 中的单一前馈网络。各专家拥有独立权重，路由器为每个 token 选择参与计算的专家；专家读取输入特征（激活）和权重，经矩阵乘法及非线性运算生成中间激活，再计算分支输出，最后按路由权重加权汇合。[1,2] 路由完成后，运行时组织专家输入，通过 grouped GEMM 内核在一次 GPU 程序调用中执行多个专家的矩阵乘法。内核将输出矩阵划分为 tile 任务，按给定顺序分派并行执行。[3] 任务按地址读取激活块和权重块。经过 L2（各计算单元共享的片上缓存）的读请求若命中，便可复用已有数据；未命中时从显存取回，并按缓存策略保留供后续任务使用。[4] 输出行对应 token、列对应特征，同一专家内的同行任务可共享激活，同列任务可共享权重；两侧数据及结果写入又共同竞争有限的 L2。最直接的方式是固定逐行遍历，先完成一行的各列，再处理下一行。[5] Wright 等的缓存感知方法[6]则让线程块（协作线程组）持续处理 tile，并将若干输出行组成行带，在带内优先计算同列任务，以兼顾两侧复用。然而，逐行遍历容易反复加载权重，固定行带也可能错过带外及后续阶段的复用机会。下面用一个例子说明这种差异。

如图 1 所示，四个 token 中，第一个由专家 A 处理，其余三个由专家 B 处理。输入 X 先与专家权重 W 相乘，经 ReLU 非线性变换生成中间激活 H ，再与权重 V 相乘得到专家输出 Y ，最后汇合为层输出 O 。图中放大的中间激活矩阵包含 B 的六个待计算块（tile）：同行的两个块共享同一 token 的输入，同列的三个块共享同一权重片段。因此，逐行计算有利于复用输入，逐列计算有利于复用权重。但缓存容量有限，读取其中一类数据可能挤出另一类数据，刚生成的中间结果也可能在后续使用前被替换。下面在相同条件下比较三种执行编排，说明 tile 顺序如何影响整个计算过程的重复读取。

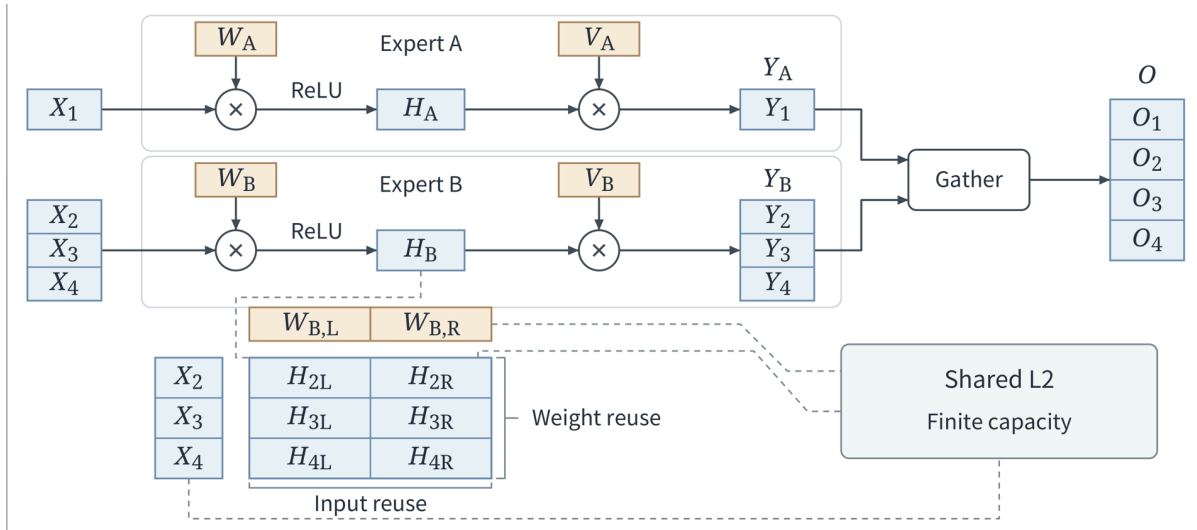


图 1: MoE 数据流与 tile 复用。

设缓存初始为空、容量为 5 个单位。第 i 个 token 的输入块 X_i 、左半与右半中间激活块 H_{iL}, H_{iR} 、专家输出块 Y_i 及层输出块 O_i 各占 1 单位； W 的每个列块占 2 单位，每个专家的 V 占 4 单位。空间不足时替换最久未访问的数据，结果写入也占用缓存。生成一个 H 块或 Y 块构成一个 tile 任务，每轮执行两个任务，先完成全部 H ，再生成全部 Y ，最后汇合，缓存内容在阶段间延续。生成 Y 时，先读取左半激活及对应的 V 片段，再读取右半；汇合则依次处理 (Y_1, Y_2) 、 (Y_3, Y_4) 。以下括号内的两个任务属于同一轮，三种计划按相同访存规则比较显存读取量。

两种对照均保持专家内部行序，并允许两个计算阶段分别选择专家次序。分组方法按专家次序连接输入行，再比较所有有效行带大小（1 至 4）；以下取各自配置范围内读取量最小的安排。[5,6,7]

逐行遍历。生成 H 时采用 $A \rightarrow B$ 次序，四轮依次计算第 1、2、3、4 行的左右两个 tile。这样每个 X_i 只需读取一次，但一行输入和两列 W 已占满缓存：写出 H_{2L}, H_{2R} 会挤出 B 的左列权重，下一行重载左列又挤出右列，导致两列反复读取。这一阶段共读取 20 单位。生成 Y 时采用 $B \rightarrow A$ 次序，两轮为 (Y_2, Y_3) 、 (Y_4, Y_1) ；第 4 行激活在被消费前已经逐出，共读取 20 单位。末轮留下的 Y_1 可供首轮汇合使用，汇合只需重读其余三个输出。整层读取量为 $20 + 20 + 3 = 43$ 。

缓存感知分组遍历。行带大小取 3，生成 H 时采用 $B \rightarrow A$ 次序：先计算 B 的左列，再计算右列，最后处理 A ，每两个连续任务组成一轮。各个 W 列块只读一次，但 B 的三个输入在左右列之间被替换，计算右列时均须重读，这一阶段读取 15 单位。生成 Y 时采用 $A \rightarrow B$ 次序，两轮为 (Y_1, Y_2) 、 (Y_3, Y_4) ： H_{1L} 命中， H_{1R} 却被本轮左半计算的读取挤出，共读取 19 单位。最后留下的 Y_3, Y_4 又在汇合先处理 Y_1, Y_2 时被替换，四个输出均须重读。整层读取量为 $15 + 19 + 4 = 38$ ，是上述分组配置族中的最小值。

更优可行编排。先完成 A ，再让 B 按右列向下、左列向上计算，四轮依次为 (H_{1L}, H_{1R}) 、 (H_{2R}, H_{3R}) 、 (H_{4R}, H_{4L}) 、 (H_{3L}, H_{2L}) 。折返处的两个任务共享 X_4 ，各个 W 列块仍只读一次，使这一阶段比前一计划少读 1 单位。随后先计算 (Y_2, Y_3) ，及时消费末轮留下的 H_{2L}, H_{3L} ，再少读 1 单位；最后计算 (Y_4, Y_1) ，使 Y_1 接上首轮汇合，又少读 1 单位。整层读取量为 $14 + 18 + 3 = 35$ 。

执行组织	生成 H 时读取	生成 Y 时读取	输出汇合读取	整层读取
逐行遍历	20	20	3	43
缓存感知分组	15	19	4	38
更优可行编排	14	18	3	35

表 1：统一简化模型下的显存读取量。三种计划计算量相同，35 为构造出的可行结果。

这个例子进一步说明，固定行带内的最好配置也可能不是更大调度空间中的局部最优。保持分组计划的 H 顺序不变，仅交换 Y 序列中相邻的 Y_2, Y_3 ，两轮变为 (Y_1, Y_3) 、 (Y_2, Y_4) ，末轮的 Y_2 可在首轮汇合中命中，整层读取量由 38 降至 37。因此，该计划在“同一阶段交换相邻两个任务”的邻域内就存在改进。基于固定遍历规则的启发式[5,6,7]尚未提供整层最优性保证，而共享输入的排序本身也有组合困难：在等大块、串行执行的 Top-8 缓存模型中，寻找最少输入加载的顺序是 NP-hard（第 3 节给出证明）。

上述差异来自同一机制：当前任务既消费缓存数据，也改变后续任务可复用的数据。本文面向单 GPU 上当前路由已知的 MoE 层，重点考虑分块预填充，即分块处理输入提示；研究离线条件下各阶段内部 tile 的联合编排，使整层完成时间最短。要从固定规则走向有依据的全局优化，需要解决以下困难。

首先，任务代价依赖此前的访问历史。同一个 tile 在不同编排下可能命中或重载，其代价不能预先固定为一个与顺序无关的数。只看相邻任务共享多少数据也不够，因为并发访问和结果写入会改变哪些数据仍在缓存中。

其次，当前较好的编排未必能接出较好的后续执行。两个执行前缀即使完成相同任务，也可能留下不同缓存内容；只比较已花费的代价，就可能提前排除后续更有利的选择。直接按专家或阶段独立优化缺少依据，而保留全部任务进度与缓存状态又会迅速扩大搜索空间。

最后，逻辑次序不能直接确定实际时间。多个任务的访问会交错，读取还可能与计算重叠；用同步强制维持某个复用次序，又可能牺牲并行度。因此，需要在实际可执行的计划中比较整层收益，同时计入运行时调度开销，不能把读取量减少直接当成延迟保证。

2. Preliminary 与问题定义

2.1 研究场景与计算流程

本文研究单 GPU 上一个 MoE 层的离线计算调度，主要面向中等规模的分块预填充，即分块处理输入提示。一个实例对应当前 batch 的一次层计算，起点为该层路由完成。此时，token 到专家的映射、汇合权重、矩阵形状、分块方式和数据地址均已给定，输入及所需专家权重已驻留显存。

计算流程。 MoE 层由路由器选择专家处理各个 token。专家读取输入激活，即 token 的特征数据，与自身权重进行矩阵乘法和非线性变换，生成中间激活，再计算专家输出；最后将同一 token 的各分支输出按路由权重加权汇合。[1,2,9] 数据整理、非线性变换和汇合均沿用既定实现。本文固定路由、数值精度、分块尺寸、数据布局及任务内部计算，只调整计算任务的执行组织。

离线设定。 当前实例的任务信息在求解前已知，先确定执行安排，再从给定的路由完成状态执行或模拟。求解过程不受即时决策时限约束，也不要求预知后续层或后续 batch 的路由。

2.2 计算任务与依赖

输出 tile 任务。 矩阵乘法的输出行对应 token，列对应输出特征。一个输出 tile 是其中的一块矩形区域，对应部分 token 的部分特征；计算该区域的完整结果构成一个任务。任务沿参与乘法的特征维度逐片读取激活和权重，完成全部累加后才得到该 tile 的输出。[10] 任务内部计算与访存次序固定。

任务集合。 用 $\mathcal{V}$ 表示从路由完成到整层输出形成所需的全部任务。除输出 tile 外，数据整理、非线性变换和最终汇合也按既定实现保留为独立任务或 tile 内部操作。调度改变其中输出 tile 的编排，其余操作仍须完成并计入整层代价。

依赖集合。 用 $\mathcal{E}$ 表示任务间必须保留的依赖，用 u 和 v 表示 $\mathcal{V}$ 中的任意两个任务。关系 $(u, v) \in \mathcal{E}$ 表示任务 v 须在 u 完成后执行。依赖既包括数据生产与消费关系，也包括固定实现要求的阶段同步。例如，生成中间激活的任务完成后，读取它的任务才能执行；专家输出生成后，相应汇合任务才能执行。若实现要求整个阶段结束后才进入下一阶段，这一同步关系也属于 $\mathcal{E}$ 。

2.3 共享 L2 与数据复用

共享缓存。 L2 是多个 GPU 计算单元共享的片上缓存，其容量记为 C ，与数据大小采用一致的计量单位。激活、权重、中间结果和输出写入共同占用这部分空间。对需要访问 L2 的请求，命中可以复用已有数据，未命中则需要从显存读取；因此，数据已驻留显存并不意味着没有重复读取开销。[4]

复用关系。 复用以相同的数据地址为依据。同一专家内，同行输出 tile 读取相同输入，同列输出 tile 读取相同权重片段。不同专家若读取原始输入的同一地址，也有机会共享缓存副本；若输入已复制到不同地址，则不能仅凭内容相同判为共享。[2,5] 当前布局和地址由此确定任务之间可利用的复用关系。

容量竞争。 新读取的数据和结果写入可能挤出尚待复用的数据，任务次序因而影响后续命中与重载。缓存替换由硬件或给定缓存模型决定，调度通过访问顺序影响缓存状态。当所需数据无法同时放入容量为 C 的缓存时，产生的替换和重读计入执行代价，不要求整层数据同时驻留 L2。

2.4 调度计划与整层代价

调度计划。 用 π 表示一个计划，规定各计算阶段内部输出 tile 的分配、遍历及跨专家编排。这里的**执行后端**指承担计算与任务分派的 GPU 程序；**工作单元**指后端接收并执行 tile 的单位，例如线程块，即一组协作线程。[3] 计划用分阶段的任务表表示：每项标识一个 tile，表项次序规定分派或遍历顺序；后端允许时还指定工作单元，否则采用后端给定的映射。

同一阶段内，不同专家的 tile 可以交错，具备条件的任务可以并行。任务表规定的是软件能够控制的分派方式，**分派顺序不等于实际完成顺序**，也不指定任意任务的精确启动时刻。

可行计划。 用 Π 表示当前实例的全部可行计划，即能够由固定后端执行、正确完成 $\mathcal{V}$ 中全部任务、保留 $\mathcal{E}$ 并满足资源限制的计划集合。可行性由任务表与后端规则检查。因此， Π 的范围由后端实际支持的分派方式确定。

整层延迟。 对任意可行计划 $\pi \in \Pi$ ，用 $T(\pi)$ 表示从给定的路由完成状态到当前 batch 的该层全部输出形成所需的时间。计时开始时离线计划已可用，但使其生效所需的运行时准备、任务表装载、索引处理和任务分发均计入 $T(\pi)$ ；数据整理、专家计算、非线性变换和最终汇合也全部计入。离线搜索与候选评估时间单独记录，不计入 $T(\pi)$ ；若在路由完成后即时求解，则求解时间还须计入对应的在线延迟。

统一评价。 比较计划时，固定 GPU 和执行后端，各计划从相同的初始缓存状态执行，并保持一致的外部干扰条件和代价规则。测量评价固定计时方式、重复次数和比较所用的统计量；模型评价明确访问交错、缓存更新、计算与搬运重叠以及运行时开销。这一规则属于问题输入。任务代价允许随缓存竞争和并发情况变化，不预设与计划无关的逐任务耗时。

2.5 问题定义

在可行集合 Π 中，使整层延迟 $T(\pi)$ 最小的计划记为 π^* ，称为**最优调度计划**。表 2 汇总本节符号。

符号	含义
$\mathcal{V}$	从路由完成到整层输出形成所需的全部任务
u, v	$\mathcal{V}$ 中的任意两个任务
$\mathcal{E}$	必要依赖集合； $(u, v) \in \mathcal{E}$ 表示 u 完成后才能执行 v
C	共享 L2 缓存容量
π	各计算阶段内部的 tile 分配、遍历及跨专家编排计划
Π	当前实例与固定执行后端下的全部可行计划
$T(\pi)$	计划 π 的整层执行延迟，包含运行时准备与调度开销
π^*	在 Π 中使 $T(\pi)$ 最小的计划

表 2：问题定义的符号。

定义 1 (MoE 层离线 tile 调度问题)。 在第 2.1—2.4 节的设定下，问题的输入、输出、约束和目标如下。

项目	形式化定义
输入	当前 batch 的完整路由结果、汇合权重、矩阵形状、数据地址及固定计算配置，由此确定任务集合 $\mathcal{V}$ 和依赖集合 $\mathcal{E}$ ，输入及权重已驻留显存；容量为 C 的共享 L2、初始缓存条件、固定 GPU 与执行后端，以及统一代价评价规则。
输出	一个最优可行计划 $\pi^* \in \Pi$ ，安排各计算阶段内部输出 tile 的分配、遍历及跨专家执行。
约束	任务完整： 每个必要任务正确完成，输出 tile 无遗漏或重复计算，使用给定地址并保持输出位置和汇合关系正确。 依赖保留： 对所有 $(u, v) \in \mathcal{E}$ ， v 在 u 完成后执行，固定阶段同步不变。 执行合法： 计划满足固定后端的并发、同步和资源限制，路由、精度、分块、布局及任务内部计算不变。

项目	形式化定义
优化目标	最小化整层延迟 $T(\pi)$ ，即从路由完成到全部层输出形成的时间，包含执行计划所需的运行时准备与调度开销。

表 3: MoE 层离线 tile 调度问题。

对应的优化形式为

$$\pi^* \in \Pi, \quad T(\pi^*) = \min_{\pi \in \Pi} T(\pi). \quad (1)$$

缓存复用的收益通过 $T(\pi)$ 体现，不另设缓存占用率或单块驻留时长目标。最优性相对于给定的 Π 和代价规则成立；采用性能模型时，结论也以该模型为准。

3. 简化缓存模型中的排序困难性

本节提取“任务共享输入且竞争有限缓存”这一排序因素，建立一个独立的简化模型。引言例子的激活共享发生在同一专家的不同输出列任务之间，还包含阶段衔接；本节则考察 Top-8 下多个专家读取同一 token 激活的共享关系，将任务串行执行，以单阶段输入加载次数为目标。下述结果分析这一受限模型的组合困难，不将其直接视为定义 1 已证困难的特殊情形。

3.1 模型与定理

每个 token-专家关联对应一个任务，依次读取该 token 的激活块、该专家的权重块，再写出独占输出块。不同 token、专家和任务输出分别使用不同地址；同一 token 的不同专家任务读取同一激活地址。输入及权重已驻留显存，任务、路由和地址显式给出。

缓存初始为空，按等大块计量取 $C = 3$ ，使用全相联 LRU：数据块可使用任意缓存位置，空间不足时替换最久未访问的块，命中也更新次序。输出写入分配缓存空间，在本阶段不再读取。任务不可拆分地串行执行，内部无其他访问；唯一决策是全部任务的排列，每项恰好执行一次，不增加预取、主动逐出或重复计算。代价为激活和权重未命中的加载次数。

定理 1. 在上述模型中，即使每个 token 恰好选择 8 个不同专家、每个专家至多接收两个 token，判断是否存在输入加载次数不超过给定整数阈值的任务排列仍为 NP-complete；最小输入加载次数的优化问题为 NP-hard。token 数与专家总数可随实例增长。

3.2 缓存关系与 Top-8 规约

缓存计数。 每个任务结束后，缓存从最旧到最新恰好为该任务的激活、权重和输出：它们是最后依次访问的三个不同块，数量恰等于容量。首任务需要两次输入加载。后续任务若共享前一任务的激活，则激活命中；若共享权重，则新激活只逐出旧激活，权重仍命中；两侧均不共享时，两块输入均需加载。不同任务不同时共享两侧。因此，

$$\text{输入加载次数} = \text{任务数} + 1 + \text{不共享输入的相邻位置数}. \quad (2)$$

本节“任务数”仅指所定义阶段的任务，不是整层 $\mathcal{V}$ 的大小；“不共享输入”指激活与权重两侧均不共享。达到“任务数加一”的阈值，当且仅当每对相邻任务共享激活或权重。

来源构造。 从平面三正则二分图上的 Hamiltonian Path 问题规约。Hamiltonian Path 是恰好经过每个顶点一次的路径，三正则指每个顶点的度数为 3。Munaro 的 Theorem 23 证明该问题为 NP-complete；其 Theorem 33 的规约证明，原图存在这样的路径，当且仅当原图逐边细分一次后所得图的线图存在这样的路径。[11] 细分一次是在每条边中插入一个顶点；线图以细分后的边为顶点，共享端点

的边在其中相邻。

将原图的一个顶点映射为一个 token，一条边映射为一个专家，顶点与边的一次关联映射为一个任务。每个 token 先选择三个专家，每个专家接收两个 token。以任务为顶点、以共享激活或权重为邻接关系得到的图，恰好就是上述线图。因此，原图存在 Hamiltonian Path，当且仅当初始任务存在相邻始终共享输入的排列。

扩展为恰好 Top-8。 对每个 token 增加五个仅接收该 token 的私有专家。新增专家均有独立权重，正常加载、计算和输出，分支权重可取正值。每个 token 由此恰好选择八个不同专家；原任务在其他 token 中仍各有一个共享权重的伙伴，新增任务则只与本 token 的任务共享激活。

初始排列若相邻始终共享输入，则每个 token 的三个原任务中至少一个处于排列内部，因为全排列只有两个端点。该任务在其他 token 中只有一个伙伴，故前后邻居至少有一个属于本 token。将五个新增任务插入这对同 token 原任务之间，全部新相邻位置仍共享激活。各 token 的插入互不冲突。

反之，从扩展后的连续共享排列中删除全部新增任务。一个新增任务的极大连续段只能通过共同激活衔接，因而属于同一 token；若两端均有原任务，两端也都共享这一激活，删除后仍可相邻。位于排列端部的段直接删除即可。由此恢复初始任务的连续共享排列。扩展保持双向等价，不要求一个 token 的全部任务连续执行。

阈值与可验证性。 扩展后任务数为原图顶点数的八倍，专家数为其十三倍除以二，构造与编码均为多项式量级。由双向等价及式 (2)，来源图存在 Hamiltonian Path，当且仅当构造实例存在加载次数不超过“任务数加一”的排列，故判定问题为 NP-hard。给定排列，可检查每个任务出现一次，并模拟每任务三次访问，在多项式时间核验代价，故问题属于 NP。优化问题若可多项式求解，比较其最优值与该阈值即可解决判定问题。因此定理成立。

3.3 时间推论与结果作用

推论 1。 若上述阶段的时间等于与排列无关的固定开销，加上一个固定正系数乘以输入加载次数，且两项系数均为可多项式计算、具有多项式编码长度的有理数，则最小阶段时间为 NP-hard。

证明。 将加载阈值“任务数加一”代入该正仿射时间关系，得到时间阈值；正系数保证两个阈值判定等价，应用定理 1 即得。固定开销可以包含已证明与排列无关的计算、输出和分发费用。每个输出首次写入，且除末任务外的脏输出均在阶段内被逐出，这两项次数也固定。证毕。

本节的 NP-hard 结论针对上述串行缓存排序模型。定义 1 还允许并行任务，并包含后续计算和运行时调度成本；其完整整层延迟的复杂度结论仍需单独建立，不能由本节直接推出。

4. Related Work

4.1 输入组织与专家计算

CUTLASS 的 grouped GEMM 调度机制和 MegaBlocks 的块稀疏实现，都将多个专家的工作组织到矩阵计算层面，以处理路由后不一致的任务形状。[3,12] ScatterMoE 将 grouped GEMM 与索引读写结合，减少部分完整输入数组的重排、复制和相应内存开销。[2] 这些方法确定了计算如何读取专家输入、如何保存分支结果，也决定了哪些访问实际指向同一地址。本文固定给定实例的布局与数据地址，在这一基础上研究任务组织。

4.2 分块、持久化与缓存感知编排

CUTLASS 通过分层分块组织共享内存和寄存器中的局部复用，并利用线程块遍历影响缓存访问。[10] Triton 的行带遍历及 Wright 等人的缓存感知持久化 grouped GEMM 进一步将工作分派与 L2 局部性结合，明确兼顾激活和权重两侧的复用。[5,6] 引言基线参照其历史实现的全局行轴映射，在统一模型中比较配置，而非复现完整 kernel 的实测性能。[7]

这些机制提供了本文所需的执行接口和有价值的编排规则。本文的问题刻画关注给定后端下各阶段的联合评价：一次编排同时改变当前复用、阶段未缓存内容和后续访问，收益最终由完整层的时间衡量。是否优于具体实现，须在相同后端和代价规则下比较。

4.3 融合与容量受限的共享数据调度

SonicMoE 将索引读取、矩阵计算和部分结果处理结合，并通过数据搬运与计算重叠改善执行。[8] 融合改变访问轨迹，重叠改变访存对时间的影响。本文将既定融合与任务内部计算作为固定条件，使候选计划在相同计算实现上比较。

Gonthier 等研究有限内存下共享输入任务的调度，联合考虑任务次序及数据加载、移除，并在 StarPU 中评估按共享数据聚合任务的策略。[13] 其实现主要面向主存与 GPU 显存之间的搬运，运行时能够显式管理设备副本。本文关注硬件管理的共享 L2，主要通过访问组织影响复用，[4] 并保留 MoE 层的数据依赖、输出写入及最终汇合。因此，两者的共享数据思想相关，但可控制的存储操作和完整评价对象不同。

参考文献

- [1] Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, et al. **Mixtral of Experts**. arXiv:2401.04088, 2024. [论文全文](#).
- [2] Shawn Tan, Yikang Shen, Rameswar Panda, Aaron Courville. **Scattered Mixture-of-Experts Implementation**. COLM, 2024. [论文全文](#).
- [3] NVIDIA. **Grouped Kernel Schedulers**. CUTLASS 官方文档. [文档](#).
- [4] NVIDIA. **CUDA C++ Programming Guide**, Release 12.4, 2024. [文档](#).
- [5] Triton. **Matrix Multiplication**. 官方教程, L2 Cache Optimizations. [教程与代码](#).
- [6] Less Wright, Adnan Hoque, Garrett Goon. **Accelerating MoE's with a Triton Persistent Cache-Aware Grouped GEMM Kernel**. PyTorch, 2025-08-18. [官方文章](#).
- [7] PyTorch / torchtitan. **cg_forward.py**. 固定提交: fd230800d2385e782f4443e24f1e45d83a6e6b0d, 2025-08-21. [历史源码](#).
- [8] Wentao Guo, Mayank Mishra, Xinle Cheng, Ion Stoica, Tri Dao. **SonicMoE: Accelerating MoE with IO and Tile-aware Optimizations**. ICLR, 2026; 本文参照扩展稿 arXiv:2512.14080v2. [论文全文](#); [会议记录](#).
- [9] Noam Shazeer. **GLU Variants Improve Transformer**. arXiv:2002.05202, 2020. [论文全文](#).
- [10] NVIDIA. **Efficient GEMM in CUDA**. CUTLASS 官方文档. [文档](#).
- [11] Andrea Munaro. **On Line Graphs of Subcubic Triangle-Free Graphs**. *Discrete Mathematics*, 340(6):1210–1226, 2017. [DOI](#); [作者稿](#). 本文使用 Theorem 23 与 Theorem 33; 恰好 Top-8 的扩展及缓存代价规约由本文给出。
- [12] Trevor Gale, Deepak Narayanan, Cliff Young, Matei Zaharia. **MegaBlocks: Efficient Sparse Training with Mixture-of-Experts**. *Proceedings of Machine Learning and Systems*, 5, 2023. [论文与出版信息](#).
- [13] Maxime Gonthier, Loris Marchal, Samuel Thibault. **Taming data locality for task scheduling under memory constraint in runtime systems**. *Future Generation Computer Systems*, 143:305–321, 2023. [DOI](#).